
Introduction to language models

1.1 Chapter overview and learning outcomes

Language modeling represents the fundamental probabilistic bedrock of contemporary Natural Language Processing (NLP) and modern AI-assisted Software Engineering. At its core, a language model formalizes the intuitive mechanism by which humans evaluate the fluency, plausibility, and likelihood of word sequences. This chapter establishes the mathematical foundations of language modeling, beginning with joint probability distributions over discrete token vocabularies, developing the unconditional decomposition provided by the probability chain rule, and examining the classic Markovian simplifying assumptions that yield statistical n -gram models.

We explore empirical transition counting matrices, trace step-by-step autoregressive text generation, analyze the three crippling pathologies of classical counting models (combinatorial sparsity V^n , finite context horizon, and semantic blindness), and derive the rigorous information-theoretic metrics used to benchmark language models: Cross-Entropy and Perplexity. Finally, we trace the evolutionary arc from Claude Shannon's 1948 foundational communication theory to Bengio's seminal 2003 neural probabilistic model, concluding with modern compute-optimal scaling laws.

Upon completing this chapter, you will be able to:

- Decompose joint probability distributions over token sequences using the probabilistic chain rule without introducing independence assumptions.
- Formalize the Markov assumption of order $n - 1$ and construct empirical n -gram transition matrices from raw corpora.
- Compute exact joint probabilities for novel sentences from estimated conditional distributions.
- Walk through the mechanical execution of autoregressive sampling, observing degenerate looping behaviors and termination dynamics.
- Quantify the three classic n -gram failure modes: parameter explosion (V^n), context horizon truncation, and orthogonal semantic isolation.
- Derive Cross-Entropy and Perplexity mathematically, mastering the base-2 versus base- e logarithmic conversions critical for academic examinations.
- Situate modern decoder-only transformers within the historical trajectory spanning Shannon (1948), Potamianos and Jelinek (1998), Bengio et al. (2003), Radford et al. (2019), and Hoffmann et al. (2022).

1.2 Probabilistic language modeling and the chain rule

Let $\mathcal{V} = \{w_1, w_2, \dots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ discrete tokens (e.g., words, sub-words, or punctuation symbols). An arbitrary natural language sentence or document of length N is represented as an ordered sequence of discrete random variables $W = (w_1, w_2, \dots, w_N) \in \mathcal{V}^N$.

Definition 1.1: Statistical language model

A *statistical language model* is a formal probability distribution $\mathbb{P}$ defined over the set of all possible discrete sequences $\mathcal{S} = \bigcup_{N=1}^{\infty} \mathcal{V}^N$. For any specific sequence $W = (w_1, \dots, w_N)$, the language model assigns a scalar probability $\mathbb{P}(W) \in [0, 1]$ satisfying the Kolmogorov probability axioms:

$$\mathbb{P}(W) \geq 0 \quad \forall W \in \mathcal{S}, \quad \text{and} \quad \sum_{S \in \mathcal{S}} \mathbb{P}(S) = 1. \quad (1.1)$$

The model maps the syntactic and semantic conventions of human language into real values on the unit interval:

- Fluent, grammatically coherent sentences receive high probability mass:

$$\mathbb{P}(\text{"this", "is", "a", "reasonable", "sentence"}) = 0.1$$

- Syntactically valid but semantically unusual or rare sentences receive noticeably lower probability:

$$\mathbb{P}(\text{"this", "is", "a", "purple", "sentence"}) = 0.01$$

- Ungrammatical, corrupted, or disjoint sequences receive negligible probability:

$$\mathbb{P}(\text{"this", "are", "a", "reasonable", "sentence"}) = 0.001$$

Evaluating the joint probability $\mathbb{P}(w_1, \dots, w_N)$ directly over the combinatorial space $\mathcal{V}^N$ is intractable because observing every conceivable multi-word sequence in a training dataset is impossible. To structure this estimation problem, we invoke the fundamental *chain rule of probability*.

Definition 1.2: Chain rule decomposition of language models

For any sequence of random variables $(w_1, w_2, \dots, w_N)$, the joint probability factors identically and unconditionally into an ordered product of conditional next-token probabilities:

$$\begin{aligned} \mathbb{P}(w_1, w_2, \dots, w_N) &= \mathbb{P}(w_1) \cdot \mathbb{P}(w_2 | w_1) \cdot \mathbb{P}(w_3 | w_1, w_2) \cdots \mathbb{P}(w_N | w_1, \dots, w_{N-1}) \\ &= \prod_{t=1}^N \mathbb{P}(w_t | w_1, \dots, w_{t-1}) = \prod_{t=1}^N \mathbb{P}(w_t | w_{<t}), \end{aligned} \quad (1.2)$$

where $w_{<t} = (w_1, \dots, w_{t-1})$ represents the prefix history preceding timestep t . For the initial token $t = 1$, the conditioning prefix is the empty sequence $\emptyset$, giving $\mathbb{P}(w_1 | w_{<1}) = \mathbb{P}(w_1)$.

Applying Equation (1.2) to our running example yields the concrete factorization:

$$\begin{aligned} \mathbb{P}(\text{this, is, a, reasonable, sentence}) &= \mathbb{P}(\text{this}) \\ &\cdot \mathbb{P}(\text{is} \mid \text{this}) \\ &\cdot \mathbb{P}(\text{a} \mid \text{this, is}) \\ &\cdot \mathbb{P}(\text{reasonable} \mid \text{this, is, a}) \\ &\cdot \mathbb{P}(\text{sentence} \mid \text{this, is, a, reasonable}). \end{aligned}$$

Reframing language modeling as conditional prediction. This mathematical decomposition marks a pivotal conceptual shift in Natural Language Processing. Instead of attempting to score complete, static documents simultaneously, the modeling focus shifts entirely to sequential, conditional next-word estimation:

Given history $w_{<t}$, estimate the categorical probability distribution over all words in $\mathcal{V}$: $\mathbb{P}(w_t = w \mid w_{<t})$.

This formulation aligns directly with the classic psychological *cloze test* (fill-in-the-blank):

“This is a reasonable _____” $\implies \mathbb{P}(\text{sentence} \mid \text{This is a reasonable}) = ?$

Every modern autoregressive language model, including GPT-4 and Claude, relies fundamentally on this conditional next-token formulation.

💡 Intuition & analogy: The core thesis: next-word prediction as full language modeling

In the opening lecture at Politecnico di Torino, Prof. Giobergia crystallized the essence of modern language modeling:

“If we can predict the next word, then we have estimated this conditional probability. And then we can plug this into the chain rule, and estimate the probability of any arbitrary sentence. This is what a language model does: nothing more, nothing less. If you understand this idea, you understand the core mechanism driving the entire course, from 1980s Markov chains to modern trillion-parameter LLMs.”

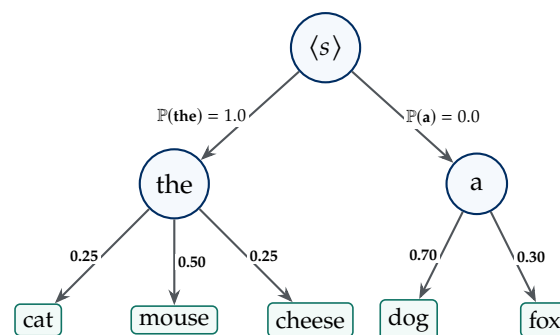


Figure 1.1: Visualizing autoregressive sequence generation as path traversal across a probability branching tree. At the root symbol $\langle s \rangle$, outgoing edges partition total probability mass across vocabulary tokens. At each node $w_{<t}$, children represent the conditional distribution $\mathbb{P}(w_t \mid w_{<t})$. The law of total probability guarantees that branch probabilities sum to unity at each decision point.

💡 Intuition & analogy: The branching probability tree

Sequence generation can be visualized geometrically as walking down a directed decision tree (Figure 1.1). At the root node $\langle s \rangle$, the model confronts the entire vocabulary $\mathcal{V}$. Choosing a token w_1 traverses an edge with probability $\mathbb{P}(w_1)$ and lands at a new conditioning state. From this state, another set of branches opens, weighted by $\mathbb{P}(w_2 | w_1)$. The joint probability of an entire sentence is simply the product of edge weights along that unique trajectory. Generating text consists of sampling a path from the root to a designated terminal token $\langle /s \rangle$.

1.3 Markov assumptions and n -gram language models

Although Equation (1.2) is mathematically exact and preserves all syntactic and semantic dependencies, computing $\mathbb{P}(w_t | w_{<t})$ directly in empirical applications presents an insurmountable combinatorial bottleneck. As the sequence index t grows, the conditioning history $w_{<t} = (w_1, \dots, w_{t-1})$ expands monotonically. Because each constituent token is drawn from a vocabulary $\mathcal{V}$ of size $V = |\mathcal{V}|$, the space of potential prefix histories preceding token t is the Cartesian product space $\mathcal{V}^{t-1}$, whose cardinality is:

$$|\mathcal{H}_t| = |\mathcal{V}|^{t-1} = V^{t-1}. \quad (1.3)$$

Even for a modest vocabulary of $V = 10,000$ words, tracking histories of length $t = 4$ requires estimating probabilities over $10,000^3 = 10^{12}$ distinct conditioning states. In any finite training corpus, almost every sequence of length greater than three occurs exactly zero times.

To make estimation mathematically tractable, the Russian mathematician Andrey Markov formulated the principle of conditional independence that now bears his name. In his landmark 1948 treatise on information theory, Claude Shannon [?] demonstrated that modeling natural language as a discrete Markov process produces increasingly coherent synthetic English as the conditioning order increases. Modern pedagogical treatments, notably by Jurafsky and Martin [?], formalize this simplification as the foundation of classical statistical natural language processing.

Definition 1.3: Markov assumption of order $n - 1$

An n -gram language model assumes that the conditional probability of the current word w_t depends exclusively on the immediately preceding $n - 1$ words, rendering it conditionally independent of all earlier discourse history:

$$\mathbb{P}(w_t | w_1, w_2, \dots, w_{t-1}) \approx \mathbb{P}(w_t | w_{t-n+1}, \dots, w_{t-1}). \quad (1.4)$$

The integer $n \geq 1$ dictates the order of the model:

- **Unigram Model ($n = 1$, Order 0):** Completely memoryless. Each word is treated as statistically independent of all past and future tokens:

$$\mathbb{P}(w_1, \dots, w_N) \approx \prod_{t=1}^N \mathbb{P}(w_t).$$

- **Bigram Model ($n = 2$, Order 1):** First-order Markov chain. Conditioning depends

exclusively on the single preceding token w_{t-1} :

$$\mathbb{P}(w_1, \dots, w_N) \approx \prod_{t=1}^N \mathbb{P}(w_t \mid w_{t-1}).$$

- **Trigram Model ($n = 3$, Order 2):** Second-order Markov chain. Conditioning depends on the preceding pair of tokens (w_{t-2}, w_{t-1}):

$$\mathbb{P}(w_1, \dots, w_N) \approx \prod_{t=1}^N \mathbb{P}(w_t \mid w_{t-2}, w_{t-1}).$$

1.3.1 Combinatorial state-space reduction

The primary mathematical virtue of the Markov assumption is that it truncates an exponentially growing state space into a fixed, stationary cardinality. Under an n -gram model, the conditioning context is always restricted to $n - 1$ words, regardless of the absolute sequence position t :

$$|\mathcal{H}^{(n)}| = |\mathcal{V}|^{n-1} = V^{n-1}. \quad (1.5)$$

Consequently, specifying the complete transition probability distribution across the entire vocabulary requires an $(n - 1)$ -dimensional conditioning tensor containing:

$$P_{\text{params}} = V^{n-1} \times (V - 1) \approx V^n \quad (1.6)$$

free parameters. Table 1.1 contrasts the unconstrained historical state space with fixed n -gram models for realistic vocabulary sizes.

Table 1.1: State-space cardinality $|\mathcal{H}|$ as a function of sequence position t versus fixed n -gram Markov orders for vocabularies $V = 10,000$ and $V = 50,000$.

Model / Order	Effective Context	Formula	States ($V = 10^4$)	States ($V = 5 \times 10^4$)
Unconstrained ($t = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Unconstrained ($t = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
Unconstrained ($t = 5$)	4 previous tokens	V^4	1.0×10^{16}	6.25×10^{18}
Unconstrained ($t = 10$)	9 previous tokens	V^9	1.0×10^{36}	1.95×10^{42}
Unconstrained ($t = 20$)	19 previous tokens	V^{19}	1.0×10^{76}	1.90×10^{89}
Unigram ($n = 1$)	0 tokens (memoryless)	$V^0 = 1$	1	1
Bigram ($n = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Trigram ($n = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
4-gram ($n = 4$)	3 previous tokens	V^3	1.0×10^{12}	1.25×10^{14}

While the unconstrained state space exceeds the total number of atoms in the observable universe ($\approx 10^{80}$) by step $t = 20$, a bigram model maintains a manageable state space of 50,000 rows. However, as derived in Section 1.6, this drastic dimensional reduction comes at the heavy cost of complete amnesia for any context preceding the immediate $n - 1$ tokens.

1.3.2 Maximum likelihood estimation (MLE)

Under Maximum Likelihood Estimation, we estimate the conditional transition probabilities directly by counting occurrences of exact token sequences in a training corpus [?]. For a generic

n -gram:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}, \dots, w_{t-1}) = \frac{C(w_{t-n+1}, \dots, w_{t-1}, w_t)}{C(w_{t-n+1}, \dots, w_{t-1})}, \quad (1.7)$$

where $C(\cdot)$ denotes the raw frequency of that exact sequence in the training text. For a bigram model ($n = 2$), Equation (1.7) specializes to:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})} = \frac{C(w_{t-1}, w_t)}{\sum_{w \in \mathcal{V}} C(w_{t-1}, w)}. \quad (1.8)$$

Normalizing each row by the unigram count of the conditioning prefix ensures that $\sum_{w \in \mathcal{V}} \mathbb{P}_{\text{MLE}}(w \mid w_{t-1}) = 1$.

1.4 Transition count matrix construction walkthrough

To understand exactly how statistical n -gram language models operate, we trace the step-by-step extraction of bigram frequencies and transition probabilities from the authoritative lecture corpus:

1. Sentence 1: *"The cat chased the mouse happily"*
2. Sentence 2: *"The mouse ate the cheese"*

Vocabulary extraction. Extracting all unique tokens (case-normalized with standard initial capitalization for clarity) yields a vocabulary of size $|\mathcal{V}| = 7$:

$$\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}.$$

Sequential bigram decomposition. We iterate through both training sentences token by token, recording each observed transition ($w_{t-1} \rightarrow w_t$):

- From Sentence 1 (*"The cat chased the mouse happily"*):
 1. (The, cat) $\implies C(\text{The, Cat}) \leftarrow 1$
 2. (cat, chased) $\implies C(\text{Cat, Chased}) \leftarrow 1$
 3. (chased, the) $\implies C(\text{Chased, The}) \leftarrow 1$
 4. (the, mouse) $\implies C(\text{The, Mouse}) \leftarrow 1$
 5. (mouse, happily) $\implies C(\text{Mouse, Happily}) \leftarrow 1$
- From Sentence 2 (*"The mouse ate the cheese"*):
 6. (The, mouse) $\implies C(\text{The, Mouse}) \leftarrow 2$
 7. (mouse, ate) $\implies C(\text{Mouse, Ate}) \leftarrow 1$
 8. (ate, the) $\implies C(\text{Ate, The}) \leftarrow 1$
 9. (the, cheese) $\implies C(\text{The, Cheese}) \leftarrow 1$

Unigram normalization counts. Summing across rows provides the total unigram occurrences $C(w_{t-1})$:

$$\begin{aligned}
 C(\text{The}) &= 4 & (\text{cat} : 1, \text{mouse} : 2, \text{cheese} : 1) \\
 C(\text{Cat}) &= 1 & (\text{chased} : 1) \\
 C(\text{Chased}) &= 1 & (\text{the} : 1) \\
 C(\text{Mouse}) &= 2 & (\text{happily} : 1, \text{ate} : 1) \\
 C(\text{Ate}) &= 1 & (\text{the} : 1) \\
 C(\text{Cheese}) &= 0 & (\text{terminal word, no outgoing transitions}) \\
 C(\text{Happily}) &= 0 & (\text{terminal word, no outgoing transitions})
 \end{aligned}$$

Full empirical transition probability matrix. Applying Equation (1.8) yields the complete 7×7 transition matrix $\mathbf{M} \in \mathbb{R}^{7 \times 7}$ where row i represents condition w_{t-1} and column j represents next token w_t . Table 1.2 presents both raw counts and normalized MLE probabilities.

Table 1.2: Empirical transition count matrix and normalized bigram conditional probabilities $\mathbb{P}(w_t | w_{t-1})$ for vocabulary $\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}$.

Condition (w_{t-1})	Ate	Cat	Chased	Cheese	Happily	Mouse	The	Row Total
Ate	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cat	0 (0.0)	0 (0.0)	1 (1.00)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1
Chased	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cheese	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Happily	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Mouse	1 (0.50)	0 (0.0)	0 (0.0)	0 (0.0)	1 (0.50)	0 (0.0)	0 (0.0)	2
The	0 (0.0)	1 (0.25)	0 (0.0)	1 (0.25)	0 (0.0)	2 (0.50)	0 (0.0)	4

1.5 Autoregressive estimation and generation

With the transition matrix $\mathbf{M}$ constructed, the language model can execute two distinct operational tasks:

1. **Sequence Probability Estimation:** Assign a likelihood score to any proposed test sentence.
2. **Autoregressive Generation:** Incrementally sample tokens to produce novel synthetic text.

1.5.1 Worked sentence probability calculation

Consider evaluating the joint probability of the sentence:

“the cat chased the cheese”

Under the first-order Markov assumption, the joint probability factors as:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = \mathbb{P}(\text{the}) \cdot \mathbb{P}(\text{cat} | \text{the}) \cdot \mathbb{P}(\text{chased} | \text{cat}) \cdot \mathbb{P}(\text{the} | \text{chased}) \cdot \mathbb{P}(\text{cheese} | \text{the}). \quad (1.9)$$

Assuming our generation begins deterministically with the initial token $\mathbb{P}(\text{the}) = 1.0$, we substitute the corresponding conditional probabilities from Table 1.2:

$$\begin{aligned}\mathbb{P}(\text{cat} \mid \text{the}) &= \frac{1}{4} = 0.25 \\ \mathbb{P}(\text{chased} \mid \text{cat}) &= 1.0 \\ \mathbb{P}(\text{the} \mid \text{chased}) &= 1.0 \\ \mathbb{P}(\text{cheese} \mid \text{the}) &= \frac{1}{4} = 0.25\end{aligned}$$

Multiplying these values step by step:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = 1.0 \times \frac{1}{4} \times 1 \times 1 \times \frac{1}{4} = \frac{1}{16} = 0.0625. \quad (1.10)$$

Despite the sequence “*the cat chased the cheese*” never occurring as an entire sentence in the training corpus, the bigram model assigns it a strictly positive probability of 6.25%. By decomposing text into local constituent pairs, n -gram models exhibit rudimentary combinatorial generalization.

1.5.2 5-step autoregressive generation trace

Now consider using the bigram model for open-ended text generation. Autoregressive sampling begins with a seed prefix, samples the next token from the resulting conditional probability distribution, appends the sampled token to the history, and repeats.

We trace the exact 5-step generation sequence recorded in the lecture slides, starting from the seed token “*the*”:

Step 1: Condition on prefix: history = (“the”).

- Active row in transition matrix: Row “The”.
- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: “**mouse**”.
- Generated prefix: “*the mouse*”.

Step 2: Condition on prefix: history = ($\dots$, “mouse”).

- Active row in transition matrix: Row “Mouse”.
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: “**ate**”.
- Generated prefix: “*the mouse ate*”.

Step 3: Condition on prefix: history = ($\dots$, “ate”).

- Active row in transition matrix: Row “Ate”.
- Candidate distribution: $\mathbb{P}(\text{the} \mid \text{ate}) = 1.00$ (deterministic transition).
- Emitted token: “**the**”.
- Generated prefix: “*the mouse ate the*”.

Step 4: Condition on prefix: history = ($\dots$, “the”).

- Active row in transition matrix: Row “The”.

- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: **“mouse”**.
- *Slide Note:* 😞 (A repetitive structural loop has emerged because the model cannot remember that it already emitted “mouse” two words ago).
- Generated prefix: *“the mouse ate the mouse”*.

Step 5: Condition on prefix: history = ($\dots$, “mouse”).

- Active row in transition matrix: Row “Mouse”.
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: **“happily”**.
- *Slide Note:* 😱 (**shocked face**) (A syntactically valid but bizarre, cannibalistic sentence is generated).
- Generated sequence: *“The mouse ate the mouse happily”*.

Because the word “happily” has zero outgoing transitions in Table 1.2, the process reaches an absorbing terminal state and generation halts naturally.

Exam insights: Classroom Q&A: absorbing terminal states and infinite loops

During the lecture walkthrough, two crucial student questions were analyzed by the instructor:

1. **Question 1: What happens after words like “happily” or “cheese”?**

In Table 1.2, terminal sentence-ending words have a row count of zero ($C(\text{Happily}) = 0$), making the conditional probability division $0/0$ mathematically undefined. In standard production language modeling, this is formalized by appending an explicit end-of-sequence token $\langle /s \rangle$ (or [EOS]) to every training sentence, guaranteeing that every word possesses a non-zero outgoing transition to either another word or $\langle /s \rangle$.

2. **Question 2: Why did greedy decoding enter an infinite cyclic loop?**

Because bigram models condition exclusively on the single preceding token w_{t-1} , once the model transitions from “ate” to “the”, it retains zero memory that it already described the mouse eating cheese two steps earlier. If greedy decoding is used ($\arg \max$), the transition path becomes fully deterministic and falls into a stationary limit cycle: *“the $\rightarrow$ mouse $\rightarrow$ ate $\rightarrow$ the $\rightarrow$ mouse $\rightarrow \dots$ ”*. Modern LLMs resolve this through long-context self-attention horizons, temperature-controlled stochastic sampling, and repetition penalties.

1.6 Fundamental limitations of statistical n -gram models

While n -gram models provided the foundation for speech recognition, statistical machine translation, and spelling correction throughout the 1980s and 1990s, they suffer from three fatal structural limitations that ultimately motivated the deep learning revolution.

1.6.1 1. Catastrophic data sparsity (V^n parameter explosion)

To condition on $n - 1$ words, an n -gram model requires storing a discrete probability table of dimensionality $|V|^n$. For realistic natural language applications:

- Vocabulary size: $|\mathcal{V}| \approx 50,000$ to 100,000 unique words.
- Bigram table ($n = 2$): $|\mathcal{V}|^2 = 50,000^2 = 2.5 \times 10^9$ parameters (approx. 10 GB at 32-bit floating point).
- Trigram table ($n = 3$): $|\mathcal{V}|^3 = 50,000^3 = 1.25 \times 10^{14}$ parameters (over 500 Terabytes).
- 4-gram table ($n = 4$): $|\mathcal{V}|^4 = 50,000^4 = 6.25 \times 10^{18}$ parameters (Exabyte scale).

Even if storage were free, the number of distinct 4-grams in all digitized human literature is dwarfed by 10^{18} . Consequently, the overwhelming majority of valid phrases never appear in training corpora. In standard MLE, any unobserved n -gram receives count $C = 0$, yielding $\mathbb{P}(w_t | w_{<t}) = 0$. By Equation (1.2), a single zero-count factor collapses the joint probability of an entire document to zero:

$$\mathbb{P}(\text{unseen word} | \text{context}) = 0 \implies \mathbb{P}(\text{Entire Book}) = 0.$$

While smoothing techniques (such as Laplace add-one, Good-Turing discounting, and Kneser-Ney backoff) alleviate zero probabilities, they reallocate probability mass heuristically without addressing the underlying representational pathology.

1.6.2 2. Finite context horizon limitations

An n -gram model is strictly blind to any token that occurred more than $n - 1$ steps in the past:

$$\mathbb{P}(w_t | w_1, w_2, \dots, w_{t-1}) \approx \mathbb{P}(w_t | w_{t-n+1}, \dots, w_{t-1}). \quad (1.11)$$

Because the conditioning window is rigidly restricted to the preceding $n - 1$ tokens, any semantic constraint, topic coherence, or syntactic agreement established earlier in the discourse is discarded. This finite context horizon causes severe memory amnesia across two critical domains:

Linguistic agreement breakdown in natural language. Consider long-range syntactic agreements where an arbitrary number of modifiers, prepositional phrases, or subordinate relative clauses separate the grammatical subject from its governing predicate:

*“The **engineers** who designed the complex distributed compiler infrastructure [...] **were** vs **was** awarded the patent.”*

If $n = 4$ (a 4-gram model), the model predicts the verb by conditioning exclusively on the three preceding tokens:

$$\mathbb{P}(\text{verb} | \text{compiler, infrastructure, [...]}).$$

Because the singular noun “*infrastructure*” is immediately adjacent to the verb position, the local 4-gram transition statistics strongly favor the singular verb “*was*”. The model has completely forgotten the true plural antecedent subject “*engineers*”, producing an ungrammatical agreement violation.

Structural and scoping breakdown in software engineering code. In programming languages, code validity requires strict, long-range determinism governed by context-free grammars and scope resolution rules:

1. **Lexical Scoping and Variable Lifecycles:** A variable declared in a function header:

```
void process_batch(std::vector<TransactionRequest>& transactions) { ...}
```

must be invoked using valid `std::vector` member methods (e.g., `.push_back()`, `.size()`) thirty lines later. An n -gram model sees only transactions. without access to the declaration, leading to syntactically invalid completions.

2. **Bracket and Block Matching:** Program code requires exact hierarchical pairing of opening and closing delimiters (`{ ...}`, `[ ...]`, `( ...)`) and control flow constructs (`try ...catch ...finally`). Because n -gram models lack internal state stacks, they frequently emit mismatched braces or premature block terminations.

These structural failures illustrate why classical statistical counting models could never serve as reliable code completion or software synthesis engines, necessitating neural architectures capable of tracking long-range relational dependencies.

1.6.3 3. Lack of semantic generalization

In classical statistical NLP, words are treated as discrete, orthogonal atomic symbols. If the model observes:

*“The **cat** chased the mouse happily”*

it gains zero statistical confidence regarding the identical sentence containing synonyms:

*“The **feline** chased the mouse happily”*

Because “*cat*” and “*feline*” are completely separate row indices in Table 1.2, observations of one do not inform the other. There is no concept of semantic distance, synonymy, or conceptual overlap.

1.7 Evaluation metrics: cross-entropy and perplexity

To benchmark and compare different language models quantitatively without executing costly downstream applications, natural language processing relies on intrinsic, information-theoretic metrics: *Cross-Entropy* and *Perplexity* [? ?].

1.7.1 Information-theoretic foundations: entropy and KL divergence

Let $\mathcal{X}$ denote a discrete sample space (the vocabulary $\mathcal{V}$), and let $P(x)$ represent the true, underlying, but inaccessible probability distribution of human language.

Definition 1.4: Shannon entropy

The *Shannon entropy* $H(P)$ measures the fundamental, irreducible uncertainty or average information content inherent in distribution P [?]:

$$H(P) = - \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) = \mathbb{E}_{x \sim P} \left[\log_2 \frac{1}{P(x)} \right]. \quad (1.12)$$

By Shannon’s source coding theorem, $H(P)$ represents the absolute theoretical lower bound on the expected number of bits required to encode a symbol emitted by P without loss.

When constructing a language model, we approximate the true distribution P with an estimated parameterized distribution Q_θ . To quantify the statistical discrepancy between the

true distribution P and our approximating model Q_θ , we employ the *Kullback-Leibler (KL) divergence*.

Definition 1.5: *Kullback-Leibler (KL) divergence*

The *KL divergence* (or relative entropy) from model distribution Q to target distribution P is:

$$D_{\text{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right) = \mathbb{E}_{x \sim P} \left[\log_2 \frac{P(x)}{Q(x)} \right]. \quad (1.13)$$

Theorem 1.1: *Gibbs' inequality*

For any two discrete probability distributions P and Q , the KL divergence is non-negative:

$$D_{\text{KL}}(P \parallel Q) \geq 0, \quad (1.14)$$

with equality $D_{\text{KL}}(P \parallel Q) = 0$ if and only if $P(x) = Q(x)$ for all $x \in \mathcal{X}$.

Proof. Applying the natural logarithm and utilizing the standard inequality $\ln(u) \leq u - 1$ for all $u > 0$ (with equality if and only if $u = 1$):

$$\begin{aligned} -D_{\text{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{Q(x)}{P(x)} \right) = \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \ln \left(\frac{Q(x)}{P(x)} \right) \\ &\leq \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) \\ &= \frac{1}{\ln 2} \left(\sum_{x \in \mathcal{X}} Q(x) - \sum_{x \in \mathcal{X}} P(x) \right) = \frac{1}{\ln 2} (1 - 1) = 0. \end{aligned}$$

Multiplying both sides by -1 reverses the inequality, yielding $D_{\text{KL}}(P \parallel Q) \geq 0$. ■

1.7.2 Cross-entropy decomposition

Expanding the logarithm in the definition of KL divergence demonstrates the fundamental relationship connecting Entropy, KL Divergence, and Cross-Entropy:

$$\begin{aligned} D_{\text{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \\ &= -H(P) + \left(- \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \right). \end{aligned} \quad (1.15)$$

Rearranging terms defines the *Cross-Entropy* $H(P, Q)$:

$$H(P, Q) \triangleq - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) = H(P) + D_{\text{KL}}(P \parallel Q). \quad (1.16)$$

Why optimizing cross-entropy optimizes KL divergence. Equation (1.16) reveals the theoretical justification for cross-entropy training:

- The true language distribution P is determined by human linguistic behavior; therefore, the true entropy $H(P)$ is a fixed, immutable physical constant.
- When we minimize cross-entropy with respect to model parameters θ :

$$\arg \min_{\theta} H(P, Q_{\theta}) = \arg \min_{\theta} (H(P) + D_{\text{KL}}(P \parallel Q_{\theta})) = \arg \min_{\theta} D_{\text{KL}}(P \parallel Q_{\theta}). \quad (1.17)$$

Minimizing cross-entropy is mathematically identical to driving the KL divergence between model Q_{θ} and the true language distribution P to zero.

1.7.3 Empirical sequence cross-entropy

Because we do not have closed-form access to the true distribution $P(x)$, we cannot compute the expectation $\mathbb{E}_{x \sim P}[\cdot]$ directly. Instead, by the Shannon-McMillan-Breiman theorem (the ergodic theorem for stationary information sources), the expected value is estimated empirically by sampling a long test sequence $W = (w_1, w_2, \dots, w_T) \sim P$:

$$\mathcal{H}(W) = -\frac{1}{T} \log_2 Q(w_1, w_2, \dots, w_T) = -\frac{1}{T} \sum_{t=1}^T \log_2 Q(w_t \mid w_{<t}). \quad (1.18)$$

Empirical cross-entropy measures the average number of bits required to compress each token of test text using our model Q .

1.7.4 Perplexity (PPL)

While cross-entropy evaluates performance on a logarithmic scale, researchers and practitioners overwhelmingly report *Perplexity* (PPL), defined as the exponentiated cross-entropy loss:

$$\text{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{-\frac{1}{T} \sum_{t=1}^T \log_2 Q(w_t \mid w_{<t})} \quad (1.19)$$

$$= \left(\prod_{t=1}^T \frac{1}{Q(w_t \mid w_{<t})} \right)^{\frac{1}{T}}. \quad (1.20)$$

Perplexity represents the reciprocal geometric mean of the token probabilities assigned by the model across the test corpus.

1.7.5 Rigorous proof: perplexity as the effective branching factor

The fundamental intuition taught across NLP literature is that perplexity quantifies the *effective branching factor* of a language model. We now formalize this property through a rigorous proof.

Theorem 1.2: *Perplexity of a uniform K -ary distribution*

Let a language model Q predict next tokens such that at each timestep t , the probability mass is distributed uniformly across exactly K equally likely candidate tokens ($Q(w_t \mid w_{<t}) = \frac{1}{K}$ for each candidate). Then the perplexity on any generated sequence W of length T is identically K :

$$\text{PPL}(W) = K. \quad (1.21)$$

Proof. Substituting the uniform prediction probability $Q(w_t | w_{<t}) = \frac{1}{K}$ into the empirical cross-entropy formula:

$$\begin{aligned}\mathcal{H}(W) &= -\frac{1}{T} \sum_{t=1}^T \log_2 \left(\frac{1}{K} \right) = -\frac{1}{T} \sum_{t=1}^T (-\log_2 K) \\ &= \frac{1}{T} (T \log_2 K) = \log_2 K.\end{aligned}$$

Exponentiating the cross-entropy:

$$\text{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{\log_2 K} = K. \quad \blacksquare$$

Physical interpretation. When a model evaluates a text corpus and achieves $\text{PPL} = 42$, it indicates that, on average, the model's predictive uncertainty at each token is mathematically equivalent to choosing uniformly at random from a shortlist of 42 equally plausible words:

- A uniform random guesser across vocabulary $\mathcal{V}$ achieves $\text{PPL} = |\mathcal{V}|$ (e.g., $\text{PPL} = 50,000$).
- An optimal oracle that predicts every token with probability 1.0 achieves the theoretical lower bound $\text{PPL} = 1.0$.
- State-of-the-art LLMs on natural language benchmarks typically achieve perplexities between 8 and 15.

Exam insights: Base-2 vs base- e logarithmic conversions and worked exam example

A recurrent point of confusion in oral and written examinations at Politecnico di Torino is mixing base-2 information theory with base- e deep learning frameworks:

Property	Information Theory (Bits)	Deep Learning / PyTorch (Nats)
Loss Formula	$\mathcal{H}_2 = -\frac{1}{T} \sum \log_2 Q(w_t w_{<t})$	$\mathcal{L}_{\text{CE}} = -\frac{1}{T} \sum \ln Q(w_t w_{<t})$
Unit	Bits per token	Nats per token
Perplexity Formula	$\text{PPL} = 2^{\mathcal{H}_2}$	$\text{PPL} = \exp(\mathcal{L}_{\text{CE}}) = e^{\mathcal{L}_{\text{CE}}}$
Conversion Identity	$\mathcal{H}_2 = \frac{\mathcal{L}_{\text{CE}}}{\ln 2} \approx \frac{\mathcal{L}_{\text{CE}}}{0.693147}$	$\mathcal{L}_{\text{CE}} = \mathcal{H}_2 \cdot \ln 2 \approx 0.693147 \cdot \mathcal{H}_2$

Worked exam calculation. Suppose a PyTorch model trained on code completion reports an evaluation loss of $\mathcal{L}_{\text{CE}} = 2.7726$ nats.

1. Direct Base- e Perplexity:

$$\text{PPL} = e^{2.7726} = \exp(2.7726) \approx 16.00.$$

2. Conversion to Information Bits:

$$\mathcal{H}_2 = \frac{2.7726}{\ln 2} = \frac{2.7726}{0.693147} = 4.00 \text{ bits/token.}$$

3. Base-2 Perplexity Verification:

$$\text{PPL} = 2^{\mathcal{H}_2} = 2^{4.00} = 16.00.$$

Common Exam Error: If a student incorrectly evaluates $2^{\mathcal{L}_{\text{CE}}} = 2^{2.7726} \approx 6.83$, they underestimate the model's true branching factor by more than 57%. Always verify whether the logarithm used is base-2 or base- e before computing perplexity.

1.8 Historical evolution of language models

Understanding modern large language models requires appreciating the 75-year evolutionary trajectory that progressively dismantled the mathematical and computational barriers of discrete symbol counting. Language modeling has evolved not merely through increases in compute or dataset scale, but through five profound conceptual paradigm shifts:

1. **From discrete orthogonal symbols to continuous geometric embeddings:** Transitioning from sparse, mutually orthogonal representations in $|\mathcal{V}|$ -dimensional space to dense vectors in $\mathbb{R}^d$ ($d \ll |\mathcal{V}|$) where topological and geometric distance encodes syntactic and semantic similarity.
2. **From rigid Markov horizons to flexible recurrent state memories:** Replacing strict $(n - 1)$ -token context cutoffs with recurrent hidden states capable of maintaining dynamic sequential memory across variable-length histories.
3. **From sequential recurrence to parallelized self-attention:** Eliminating step-by-step sequential bottlenecks in favor of pairwise all-to-all attention mechanisms that compress the maximum interaction path length between any two tokens in a sequence to $O(1)$.
4. **From task-specific architectures to universal pre-trained foundation models:** Shifting from designing specialized models trained from scratch for each separate task to universal autoregressive base models capable of general-purpose in-context learning.
5. **From passive token predictors to aligned, agentic reasoning systems:** Augmenting next-token predictors with post-training alignment, test-time reasoning scaffolding, and interactive tool-use execution loops for complex software engineering tasks.

1.8.1 Chronological roadmap of language modeling

Figure 1.2 synthesizes the macro-historical progression of language modeling from Claude Shannon’s foundational 1948 information theory to modern autonomous software engineering agents.

1.8.2 1948–1980s: statistical information theory and Markov n-grams

The conceptual origin of language modeling stems from Claude Shannon’s 1948 landmark paper “*A Mathematical Theory of Communication*” [?]. Shannon treated written human language as a discrete stochastic process generated by an underlying stationary Markov source over a finite alphabet.

Shannon introduced the *Shannon game*—a pedagogical thought experiment where a human or an automated statistical predictor attempts to guess the next letter in an unknown sequence of text given the preceding letters. Shannon demonstrated that as the conditioning context length increases from zero-order (independent uniform character selection) to unigram (individual character frequencies), bigram (pairs), and trigram (triplets) models, the generated text rapidly converges toward natural English orthography and grammar.

In this framework, the conditional probability of token w_t given context $w_1, \dots, w_{t-1}$ is simplified via an order- $(n - 1)$ Markov assumption:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}^{t-1}) = \frac{C(w_{t-n+1}^t)}{C(w_{t-n+1}^{t-1})}, \quad (1.22)$$

where $C(\cdot)$ denotes the empirical frequency count observed in a training corpus.

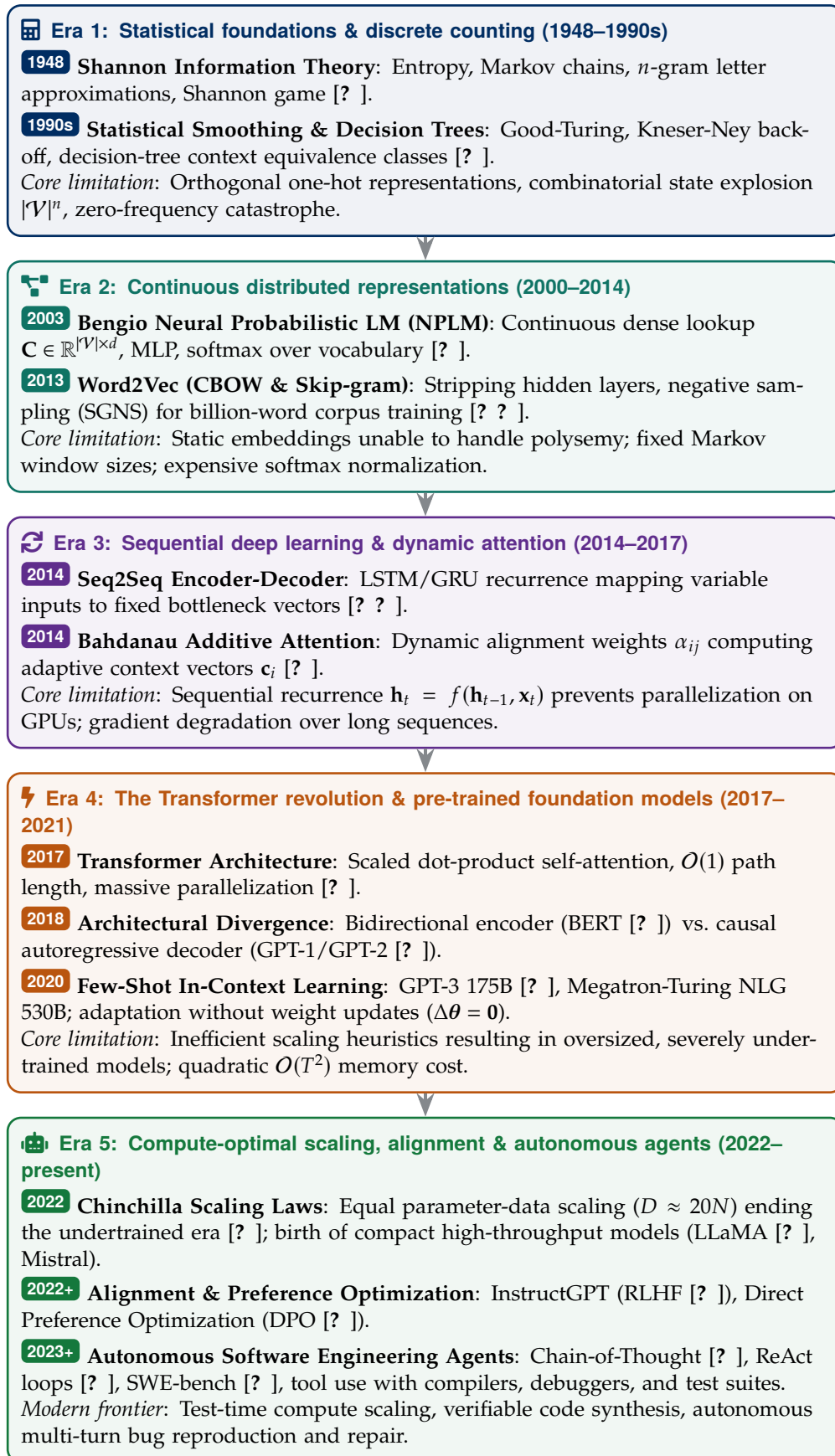


Figure 1.2: Chronological timeline of language model evolution from 1948 to the present.

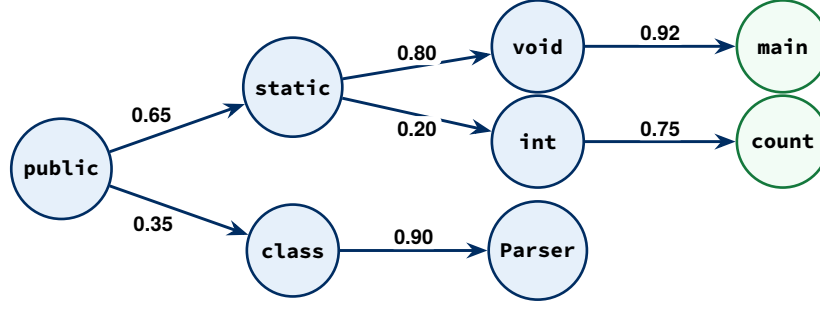


Figure 1.3: Markov chain state transition graph for software tokens.

Mathematical and operational bottlenecks. While mathematically transparent and computationally trivial to estimate via hash tables, statistical n -gram models suffer from catastrophic structural limitations:

1. **Combinatorial State Explosion:** The number of possible transition parameters scales exponentially as $|\mathcal{V}|^n$. For a modest software vocabulary of $|\mathcal{V}| = 50,000$ tokens, a 4-gram model entails $|\mathcal{V}|^4 = 6.25 \times 10^{18}$ potential state combinations, rendering full parameter estimation mathematically intractable.
2. **The Zero-Frequency Catastrophe:** If a valid code sequence (e.g., `public static void run()`) never appeared verbatim in the training set, $C(w_{t-n+1}^t) = 0 \implies P(w_t | w_{t-n+1}^{t-1}) = 0$. When evaluating test corpora, a single zero probability assigns infinite perplexity ($\text{PPL} = \infty$) to the entire document.
3. **Orthogonality of Surface Symbols:** In discrete token space, the tokens `int`, `long`, and `short` are represented as completely independent, orthogonal basis vectors. The statistical model possesses zero capacity to transfer learned syntactic patterns (such as `int x = 0;`) to semantically related tokens (`long x = 0;`).

1.8.3 1990s: statistical smoothing and decision-tree language models

Throughout the 1990s, statistical language modeling was extensively refined to address the zero-frequency barrier and structural sparsity. Researchers developed two primary paradigms: statistical smoothing (interpolated backoff) and decision-tree context clustering.

Smoothing and discounting. Rather than assigning zero probability to unseen sequences, smoothing algorithms reserve a fraction of the probability mass from frequent events and redistribute it to unseen combinations. The pinnacle of this era was *Modified Kneser-Ney smoothing*, which combines absolute discounting with a novel *continuation probability*:

$$P_{\text{KN}}(w_t | w_{t-n+1}^{t-1}) = \frac{\max(C(w_{t-n+1}^t) - d, 0)}{C(w_{t-n+1}^{t-1})} + \lambda(w_{t-n+1}^{t-1})P_{\text{KN}}(w_t | w_{t-n+2}^{t-1}), \quad (1.23)$$

where $d \in (0, 1)$ is an absolute discount parameter, $\lambda(w_{t-n+1}^{t-1}) = \frac{d}{C(w_{t-n+1}^{t-1})} \cdot |\{w : C(w_{t-n+1}^{t-1}, w) > 0\}|$ is a normalizing interpolation weight, and the lower-order continuation probability measures how versatile word w_t is across diverse historical contexts:

$$P_{\text{continuation}}(w_t) = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{\sum_{w'} |\{w_{t-1} : C(w_{t-1}, w') > 0\}|}. \quad (1.24)$$

Decision-tree language models. In parallel, Potamianos and Jelinek (1998) [?] introduced decision-tree language models to overcome the rigid Markov window. Rather than conditioning probability on an exact sequence of $n - 1$ tokens, a binary decision tree asks a sequence of syntactic and lexical questions about the historical context $\mathcal{H} = (w_{t-1}, w_{t-2}, \dots)$.

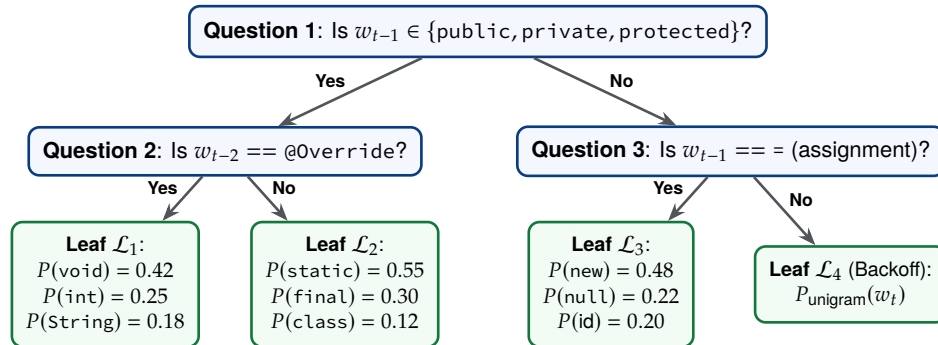


Figure 1.4: Decision tree context partitioning for conditional language modeling.

As illustrated in Figure 1.4, context histories that share identical predictive characteristics are clustered into the same leaf node $\mathcal{L}_k$, yielding conditional distributions $P(w_t | \mathcal{L}_k)$. While this reduced parameter count compared to flat tables, the model remained fundamentally bound to discrete categorical splitting without true semantic interpolation.

1.8.4 2000–2003: Bengio’s neural probabilistic language model

In their seminal 2003 paper “A Neural Probabilistic Language Model”, Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin [?] permanently transformed NLP by introducing continuous distributed representations to defeat the curse of dimensionality.

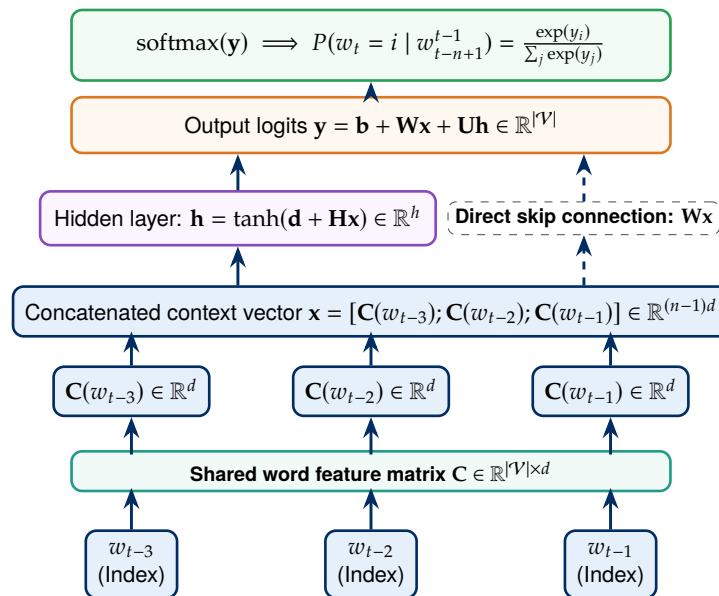


Figure 1.5: Bengio neural probabilistic language model architecture.

Architecture and mathematical formulation. As detailed in Figure 1.5, Bengio et al. decoupled representation learning from probability estimation across four stages:

1. **Continuous Embedding Matrix:** Each word $w \in \mathcal{V}$ is mapped to a continuous column

vector $\mathbf{C}(w) \in \mathbb{R}^d$ via a shared embedding matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$, with $d \ll |\mathcal{V}|$ (typically $d \in [30, 100]$ in 2003).

2. **Context Vector Concatenation:** The embeddings of the preceding $n - 1$ words are concatenated into a joint context vector:

$$\mathbf{x} = [\mathbf{C}(w_{t-n+1}); \mathbf{C}(w_{t-n+2}); \dots; \mathbf{C}(w_{t-1})] \in \mathbb{R}^{(n-1)d}. \quad (1.25)$$

3. **Non-linear Hidden Layer with Direct Skip Connections:** Context vector $\mathbf{x}$ is passed through a hyperbolic tangent hidden layer $\mathbf{h} = \tanh(\mathbf{d} + \mathbf{H}\mathbf{x})$, where $\mathbf{H} \in \mathbb{R}^{h \times (n-1)d}$ and $\mathbf{d} \in \mathbb{R}^h$. The network also incorporates an optional linear direct connection from input to output:

$$\mathbf{y} = \mathbf{b} + \mathbf{W}\mathbf{x} + \mathbf{U}\mathbf{h}, \quad (1.26)$$

where $\mathbf{U} \in \mathbb{R}^{|\mathcal{V}| \times h}$ projects the hidden state to vocabulary logits, $\mathbf{W} \in \mathbb{R}^{|\mathcal{V}| \times (n-1)d}$ is the direct skip matrix, and $\mathbf{b} \in \mathbb{R}^{|\mathcal{V}|}$ is the output bias.

4. **Softmax Multi-Class Output Head:** Unnormalized logits $\mathbf{y}$ are normalized into a proper conditional probability distribution over the vocabulary:

$$P(w_t = i \mid w_{t-n+1}^{t-1}) = \frac{\exp(y_i)}{\sum_{j=1}^{|\mathcal{V}|} \exp(y_j)}. \quad (1.27)$$

Smooth generalization and the curse of dimensionality. Because the function mapping $\mathbf{x} \mapsto P(w_t \mid \mathbf{x})$ is continuous and differentiable, words that occupy neighboring positions in $\mathbb{R}^d$ (e.g., $\mathbf{C}(\text{cat}) \approx \mathbf{C}(\text{feline})$, or $\mathbf{C}(\text{int}) \approx \mathbf{C}(\text{short})$) produce virtually identical activation vectors $\mathbf{h}$ and logit outputs $\mathbf{y}$. Consequently, seeing a sentence like “the feline climbed the tree” during training automatically increases the probability of “the cat climbed the tree”, even if the latter sequence never appeared in the corpus.

Computational bottleneck. The fatal computational bottleneck of Bengio’s NPLM lay in the output projection matrix $\mathbf{U} \in \mathbb{R}^{|\mathcal{V}| \times h}$ and the denominator of the softmax. For a vocabulary $|\mathcal{V}| = 100,000$ and hidden size $h = 500$, computing the output layer required 5×10^7 floating-point operations *per single token step*. On early-2000s CPU hardware, training on modest corpora took weeks, limiting practical applications.

External deep dive: From Shannon to Bengio: Karpathy’s Makemore perspective

In his pedagogical series *Makemore*, Andrej Karpathy demonstrates that a bigram language model can be implemented in two distinct ways that perfectly illuminate the shift from statistical counting to neural estimation:

1. **The Classical Counting Approach:** Iterate over all adjacent character pairs (w_{t-1}, w_t) in a training corpus, populate a 27×27 frequency matrix $\mathbf{C}$, normalize each row by its row-sum to obtain transition probability matrix $\mathbf{M}_{ij} = \frac{C_{ij}}{\sum_k C_{ik}}$, and generate tokens using categorical sampling: $w_t \sim \text{Categorical}(\mathbf{M}_{w_{t-1}, :})$.
2. **The Gradient-Based Neural Approach:** Encode token w_{t-1} as a one-hot vector $\mathbf{x} \in \{0, 1\}^{27}$, pass it through a single linear layer parameterized by weight matrix $\mathbf{W} \in \mathbb{R}^{27 \times 27}$ to compute logits $\mathbf{z} = \mathbf{x}\mathbf{W}$, evaluate probabilities via softmax $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$,

and minimize the negative log-likelihood cross-entropy loss:

$$\mathcal{L}(\mathbf{W}) = - \sum_{t=1}^T \log \hat{y}_{t,\text{target}}. \quad (1.28)$$

The Unifying Mathematical Duality: When optimized to theoretical convergence via gradient descent without weight regularization, the weights $\mathbf{W}$ of the single-layer neural network converge exactly to the natural logarithm of the normalized statistical counts:

$$\lim_{t \rightarrow \infty} \mathbf{W}_{ij} = \ln \mathbf{M}_{ij} + C_i, \quad (1.29)$$

where C_i is an arbitrary row-wise constant that cancels out during softmax normalization. Neural networks do not replace statistical count estimation; rather, they generalize count optimization into continuous, differentiable parameter spaces capable of hierarchical non-linear composition.

1.8.5 2013–2014: efficient distributed representations and Word2Vec

A decade after Bengio’s NPLM, Tomas Mikolov and colleagues at Google (2013) [? ?] identified that the primary impediment to scaling neural representations was the non-linear hidden layer. By discarding the tanh layer entirely, Mikolov et al. demonstrated that high-quality distributed representations could be trained across billions of tokens in hours on commodity hardware, introducing the *Word2Vec* framework.

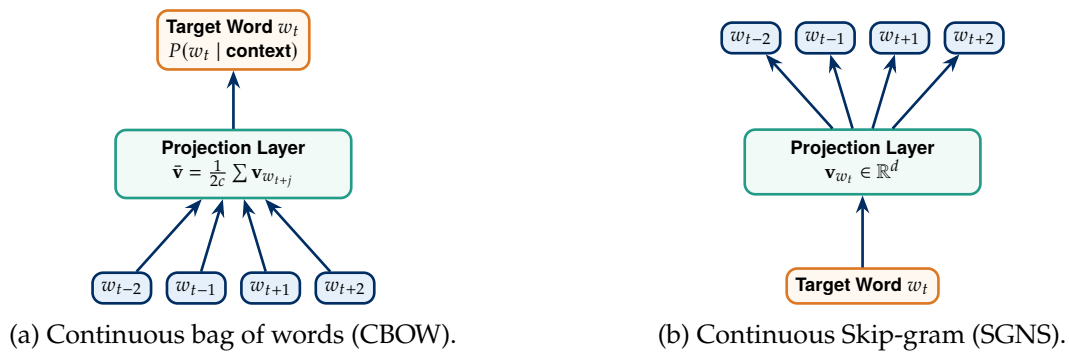


Figure 1.6: Continuous bag of words (CBOW) versus Skip-gram architectures in Word2Vec.

Word2Vec introduced two complementary architectures, shown in Figure 1.6:

1. **Continuous Bag of Words (CBOW):** Predicts the target token w_t conditioned on surrounding context words within a symmetric window $[-c, +c]$. The context embeddings are simply averaged, discarding word order:

$$\bar{\mathbf{v}} = \frac{1}{2c} \sum_{-c \leq j \leq c, j \neq 0} \mathbf{v}_{w_{t+j}}. \quad (1.30)$$

2. **Continuous Skip-Gram:** Inverts the CBOW objective by using the target token w_t to predict surrounding context tokens w_{t+j} within $[-c, +c]$.

Negative sampling (SGNS). To bypass the computationally intractable softmax denominator over $|\mathcal{V}|$, Mikolov et al. formulated *Skip-Gram with Negative Sampling* (SGNS). SGNS reframes

language modeling from a multi-class probability task into a set of binary logistic regressions: distinguishing true observed co-occurrences (w_t, c) from K randomly sampled noise pairs $(w_t, w_{n,k})$:

$$\mathcal{L}_{\text{SGNS}}(\mathbf{v}_w, \mathbf{v}'_c) = \log \sigma(\mathbf{v}'_c^\top \mathbf{v}_w) + \sum_{k=1}^K \mathbb{E}_{w_{n,k} \sim P_n} \left[\log \sigma(-\mathbf{v}'_{w_{n,k}}^\top \mathbf{v}_w) \right], \quad (1.31)$$

where $\mathbf{v}_w \in \mathbb{R}^d$ is the center word vector, $\mathbf{v}'_c \in \mathbb{R}^d$ is the context vector, $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid activation, and $P_n(w) \propto U(w)^{3/4}$ is a smoothed unigram noise distribution that slightly elevates the sampling frequency of rare words relative to stopwords.

Emergence of linear semantic vector arithmetic. SGNS produced clean vector spaces where linear geometric displacement corresponds to semantic and syntactic relationships:

$$\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} + \mathbf{v}_{\text{Woman}} \approx \mathbf{v}_{\text{Queen}}. \quad (1.32)$$

In software engineering, similar vector arithmetic emerged over AST tokens and code identifiers:

$$\mathbf{v}_{\text{ArrayList}} - \mathbf{v}_{\text{Java}} + \mathbf{v}_{\text{Python}} \approx \mathbf{v}_{\text{list}}. \quad (1.33)$$

Bottlenecks. Despite its brilliance, Word2Vec had critical deficiencies:

1. **Static, Context-Independent Embeddings (Polysemy Failure):** Each token w receives exactly one fixed vector $\mathbf{v}_w \in \mathbb{R}^d$. In natural language, “bank” (financial institution vs. river bank) cannot be disambiguated. In software engineering, the keyword `class` possesses identical static vectors regardless of whether it defines a Java object, a CSS style selector, or a C++ template parameter.
2. **Lack of Autoregressive Generative Capability:** Word2Vec produces static vector spaces for downstream classifiers; it cannot generate sequences or predict complex multi-token programs.

1.8.6 2014–2017: recurrent sequence models and Seq2Seq attention

To model variable-length sequences without rigid Markov window cutoffs, the field adopted Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTM, Hochreiter and Schmidhuber 1997 [?]), and Gated Recurrent Units (GRU, Cho et al. 2014 [?]).

Recurrent hidden states. In an autoregressive RNN language model, the conditioning history is compressed into a dynamic continuous hidden state vector $\mathbf{h}_t \in \mathbb{R}^h$ updated sequentially at each token step:

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h), \quad (1.34)$$

where $\mathbf{x}_t = \mathbf{C}(w_t) \in \mathbb{R}^d$ is the input word vector, $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$ is the recurrent transition matrix, $\mathbf{W}_{xh} \in \mathbb{R}^{h \times d}$ is the input projection, and f is a gating activation (such as the LSTM forget/input/output cell update). The next-token distribution is evaluated from $\mathbf{h}_t$:

$$P(w_{t+1} | w_1, \dots, w_t) = \text{softmax}(\mathbf{W}_{\text{out}}\mathbf{h}_t + \mathbf{b}_{\text{out}}). \quad (1.35)$$

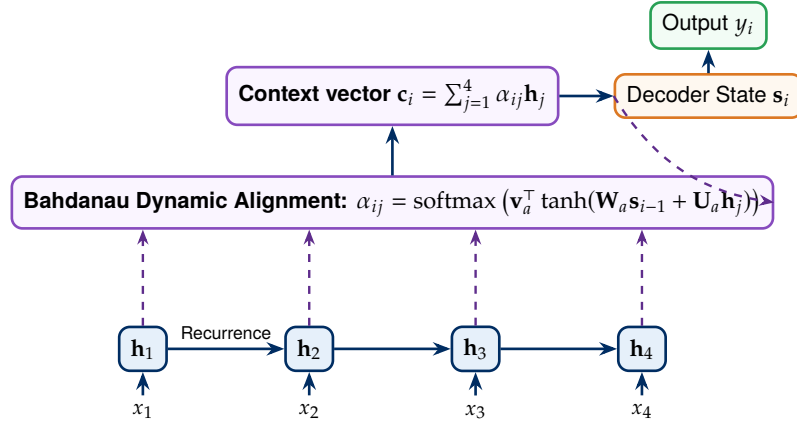


Figure 1.7: Unrolled recurrent sequence model with dynamic Bahdanau attention alignment.

Sequence-to-sequence and the Bahdanau attention breakthrough. For conditional sequence tasks (e.g., Code Summarization or Neural Machine Translation), Sutskever et al. (2014) [?] introduced the *Sequence-to-Sequence (Seq2Seq)* encoder-decoder framework. However, compressing an entire source code method containing hundreds of tokens into a single static bottleneck vector $\mathbf{h}_T$ caused performance to degrade rapidly for sequences longer than 20 tokens.

To resolve this bottleneck, Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio (2014) [?] invented *Additive Dynamic Attention*, illustrated in Figure 1.7. Rather than compressing the source into a fixed vector, the decoder computes a dynamic alignment score e_{ij} comparing current decoder state $\mathbf{s}_{i-1}$ with each encoder hidden state $\mathbf{h}_j$:

$$e_{ij} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j), \quad (1.36)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \quad (1.37)$$

$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j. \quad (1.38)$$

The dynamic context vector $\mathbf{c}_i$ allows the model to look directly back at the relevant parts of the input program, establishing the foundation of modern attention mechanisms.

The sequential computational barrier. Despite solving the informational bottleneck, recurrent models hit a hard hardware execution barrier: **sequential temporal dependency**. Because $\mathbf{h}_t$ cannot be computed until $\mathbf{h}_{t-1}$ is evaluated, training cannot be parallelized across the time dimension T . GPUs—which excel at massive parallel matrix operations—are forced into sequential execution during RNN training. Furthermore, gradients backpropagating through time (BPTT) across hundreds of steps suffer from exponential dissipation or explosion, rendering modeling across repository-scale context windows impossible.

1.8.7 2017: the Transformer revolution and self-attention

In June 2017, Ashish Vaswani and a team at Google Brain published “*Attention Is All You Need*” [?]. They introduced the **Transformer** architecture by completely eliminating recurrence and convolutions, relying entirely on self-attention mechanisms.

Mathematical mechanics of scaled dot-product attention. As shown in Figure 1.8a, given an input sequence representation $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, linear projections produce Queries (**Q**), Keys (**K**),

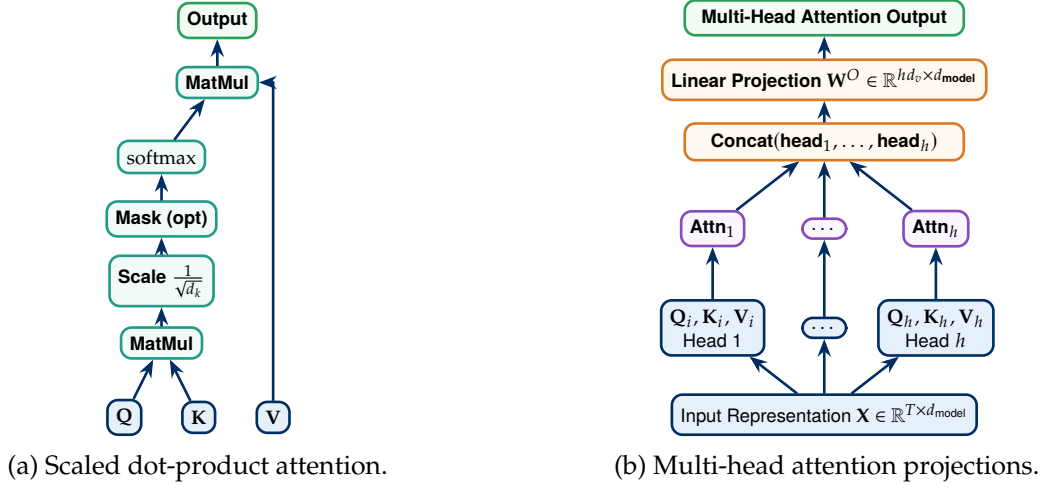


Figure 1.8: Scaled dot-product attention and multi-head attention mechanisms.

and Values ($\mathbf{V}$):

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V, \quad (1.39)$$

where $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. The attention map is computed across all token pairs simultaneously via matrix multiplication:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V}, \quad (1.40)$$

where $\frac{1}{\sqrt{d_k}}$ prevents vanishing gradients in the softmax for large inner products, and $\mathbf{M} \in \{0, -\infty\}^{T \times T}$ is an optional attention mask.

Multi-head attention (MHA). To allow the model to jointly attend to information from different representation subspaces at different positions (e.g., one head tracking variable definitions, another tracking function calls, and a third tracking syntax parentheses), Multi-Head Attention linearly projects queries, keys, and values h times in parallel:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad (1.41)$$

$$\text{where } \text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V). \quad (1.42)$$

Why Transformers conquered NLP and software engineering.

1. **$O(1)$ Path Length:** In an RNN, signal from token 1 must pass through T recurrent multiplications to reach token T . In a Transformer, every token attends directly to every other token in exactly $O(1)$ operations, eliminating vanishing gradient dissipation.
2. **Total Hardware Parallelization:** Because there are no sequential temporal dependencies across sequence length T during training, the entire attention matrix $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{T \times T}$ is evaluated as a single dense tensor contraction on GPU tensor cores.
3. **Trade-off:** Memory and computational complexity scale quadratically with sequence length: $O(T^2 \cdot d_{\text{model}})$, which motivated modern KV-caching, FlashAttention, and sparse context optimizations.

1.8.8 2018–2019: pre-trained foundation models and architectural divergence

With the Transformer established as a highly scalable backbone, NLP shifted from training task-specific models with static Word2Vec embeddings to the **pre-train then fine-tune** paradigm. Models were pre-trained on massive unlabeled web corpora using self-supervised objectives, then fine-tuned with task-specific heads.

This era witnessed a major architectural divergence into two primary branches, illustrated in Figure 1.9:

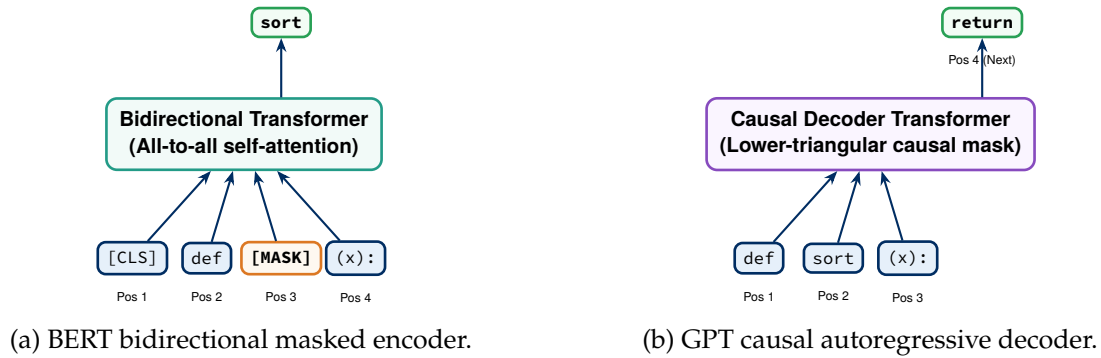


Figure 1.9: Bidirectional masked representation (BERT) versus causal autoregressive generation (GPT).

1. Bidirectional Encoders (BERT, Devlin et al. 2019 [?]):

- *Mechanism:* Tokens attend bidirectionally to all past and future positions in the sequence. Pre-trained via **Masked Language Modeling (MLM)**, where 15% of tokens are replaced with a special [MASK] token, and the network predicts the masked words:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(w_i | \tilde{\mathbf{w}}). \quad (1.43)$$

- *Software Engineering Dominance:* Bidirectional encoders excel at representation and discriminative tasks, powering tools like CodeBERT [?] for vulnerability detection, code search, defect prediction, and clone detection.
- *Limitation:* BERT cannot naturally generate variable-length code completions because bidirectional attention creates circular dependencies during sequential left-to-right decoding.

2. Autoregressive Decoders (GPT-1 & GPT-2, Radford et al. 2019 [?]):

- *Mechanism:* Retains strictly causal (unidirectional) attention by enforcing a lower-triangular causal attention mask:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i, \\ -\infty & \text{if } j > i. \end{cases} \quad (1.44)$$

The model optimizes the standard autoregressive language modeling objective:

$$\mathcal{L}_{\text{AR}} = - \sum_{t=1}^T \log P(w_t | w_1, \dots, w_{t-1}). \quad (1.45)$$

- *Radford’s Core Insight (Unsupervised Multitask Learners)*: Radford et al. demonstrated that scaling decoder-only models trained on massive, diverse web text (WebText) produced zero-shot multitask capabilities without task-specific architectural heads or fine-tuning. The decoder-only architecture became the undisputed foundation for generative AI.

1.8.9 2020–2021: extreme scale and few-shot in-context learning

In 2020, Tom Brown and colleagues at OpenAI published “*Language Models are Few-Shot Learners*” [?], introducing GPT-3 (175 billion parameters). GPT-3 revealed that scaling autoregressive decoders by an order of magnitude unlocked qualitative emergent behaviors, most notably **In-Context Learning (ICL)**.

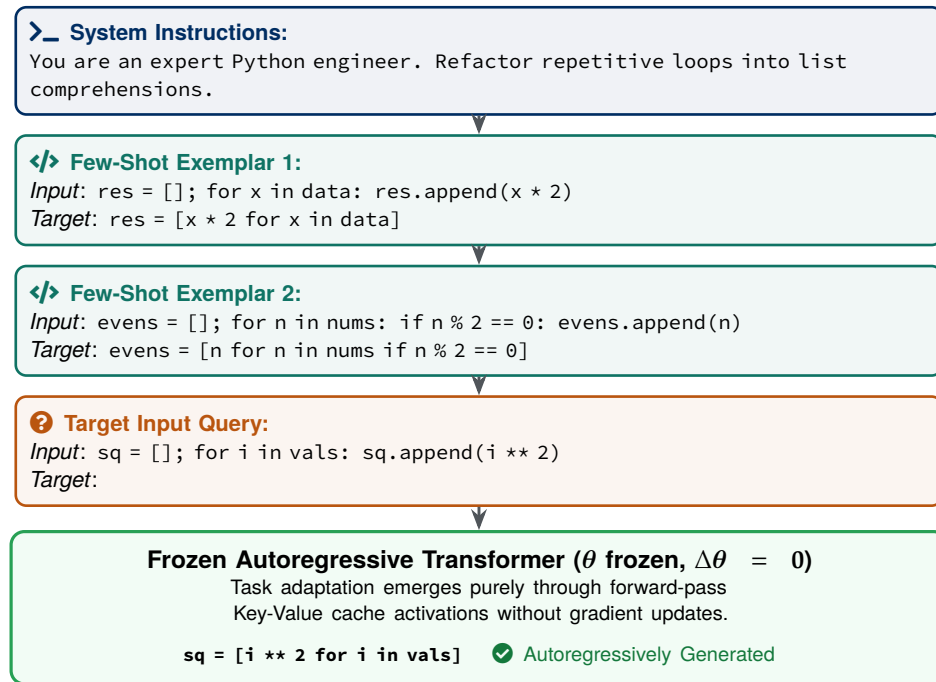


Figure 1.10: Anatomy of in-context few-shot learning within a frozen foundation model.

Mathematical formalization of in-context learning. Prior to GPT-3, adapting a model to a downstream task $\mathcal{T}$ required computing gradients and modifying parameters: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{T}}$. In contrast, In-Context Learning conditions a completely frozen parameter set θ on a prompt containing K input-output exemplar demonstrations $\mathcal{D}_{\text{demo}} = \{(x^{(1)}, y^{(1)}), \dots, (x^{(K)}, y^{(K)})\}$ and a target query x :

$$P(y \mid x, \mathcal{D}_{\text{demo}}; \theta) = \prod_{k=1}^{|y|} P(y_k \mid x_{\text{prompt}}, \mathcal{D}_{\text{demo}}, y_{<k}; \theta), \quad \text{with } \Delta\theta = 0. \quad (1.46)$$

As illustrated in Figure 1.10, task adaptation occurs purely during the forward pass through the attention mechanism: self-attention across the exemplar activations acts as an implicit, gradient-free meta-optimization process.

The parameter scaling arms race and the undertraining crisis. Following GPT-3, the industry initiated an aggressive scaling arms race, culminating in dense models like Megatron-Turing NLG 530B. However, these models were trained following the scaling laws proposed by Kaplan

et al. (2020) [?], which recommended scaling parameters significantly faster than training data. Consequently, models like GPT-3 (175B parameters trained on only 300B tokens) were severely **undertrained** relative to their parameter capacity, resulting in enormous inference latency and compute inefficiency.

1.8.10 2022: compute-optimal scaling laws and the Chinchilla shift

In March 2022, Jordan Hoffmann and colleagues at DeepMind published “*Training Compute-Optimal Large Language Models*” [?], introducing the **Chinchilla** scaling laws and fundamentally correcting the trajectory of modern AI.

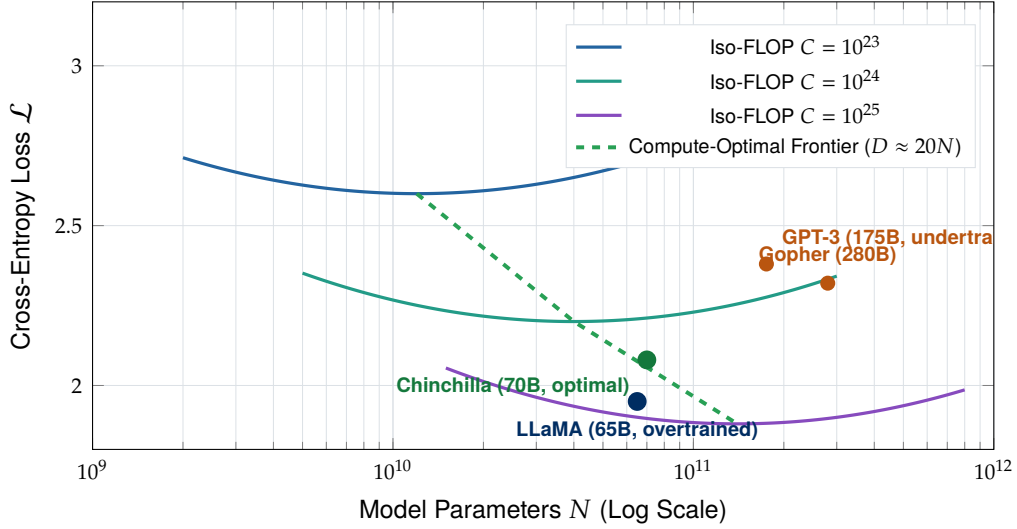


Figure 1.11: Compute-optimal scaling frontier and iso-FLOP loss trajectories.

Mathematical derivation of compute optimality. Given a fixed compute budget $C \approx 6ND$ floating-point operations (where N is parameter count and D is training token count), the cross-entropy loss is modeled as a power-law parametric surface:

$$\mathcal{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (1.47)$$

where E is irreducible entropy, A, B are scaling coefficients, and $\alpha \approx 0.34, \beta \approx 0.28$ are empirical exponents.

To find the optimal allocation of compute between model parameters N and dataset size D , we formulate the constrained optimization problem:

$$\min_{N, D} \mathcal{L}(N, D) \quad \text{subject to} \quad 6ND = C. \quad (1.48)$$

Substituting $D = \frac{C}{6N}$ into Equation (1.47) and taking the derivative with respect to N :

$$\frac{\partial \mathcal{L}}{\partial N} = -\alpha A N^{-\alpha-1} + \beta B \left(\frac{C}{6}\right)^{-\beta} N^{\beta-1} = 0. \quad (1.49)$$

Solving for the optimal parameter count N_{opt} and token count D_{opt} :

$$N_{\text{opt}} \propto C^{\frac{\beta}{\alpha+\beta}} \approx C^{0.45}, \quad D_{\text{opt}} \propto C^{\frac{\alpha}{\alpha+\beta}} \approx C^{0.55}. \quad (1.50)$$

The Chinchilla revelation: $D \approx 20N$. Because $\frac{\beta}{\alpha+\beta} \approx 0.45 \approx \frac{\alpha}{\alpha+\beta} \approx 0.55$, Hoffmann et al. proved that **parameters N and training tokens D should be scaled in equal proportion**. For compute-optimal training, a model requires approximately 20 tokens per parameter:

$$D_{\text{opt}} \approx 20N. \quad (1.51)$$

To validate this theory, DeepMind trained **Chinchilla** (70 billion parameters on 1.4 trillion tokens) using the exact same compute budget as Gopher (280 billion parameters on 300 billion tokens). Despite possessing only one-fourth the parameters, Chinchilla uniformly outperformed Gopher, GPT-3 (175B), and Megatron-Turing NLG (530B) across virtually all benchmarks, as plotted in Figure 1.11.

The modern inference-efficient paradigm: over-training. Chinchilla’s compute-optimal laws determine training efficiency. However, in production software engineering systems, models are evaluated billions of times during inference. Meta AI’s LLaMA (Touvron et al. 2023 [?]) took this insight a step further: by intentionally **over-training** compact models (training a 7B or 13B model on 1.4T to 2T tokens, and later LLaMA-3 on 15T tokens), the training compute exceeds the Chinchilla minimum, but downstream inference latency, memory footprint, and deployment costs drop by up to 80%. This gave birth to the modern open-source LLM ecosystem.

1.8.11 2022–present: alignment, reasoning, and software engineering agents

The latest evolutionary frontier marks the transition from passive text generators to goal-directed, autonomous reasoning and software engineering agents. A base foundation model trained solely on web tokens predicts statistical continuations; it does not natively follow instructions, verify factual correctness, or debug software.

Transforming base models into coding assistants requires a multi-stage post-training pipeline, illustrated in Figure 1.12:

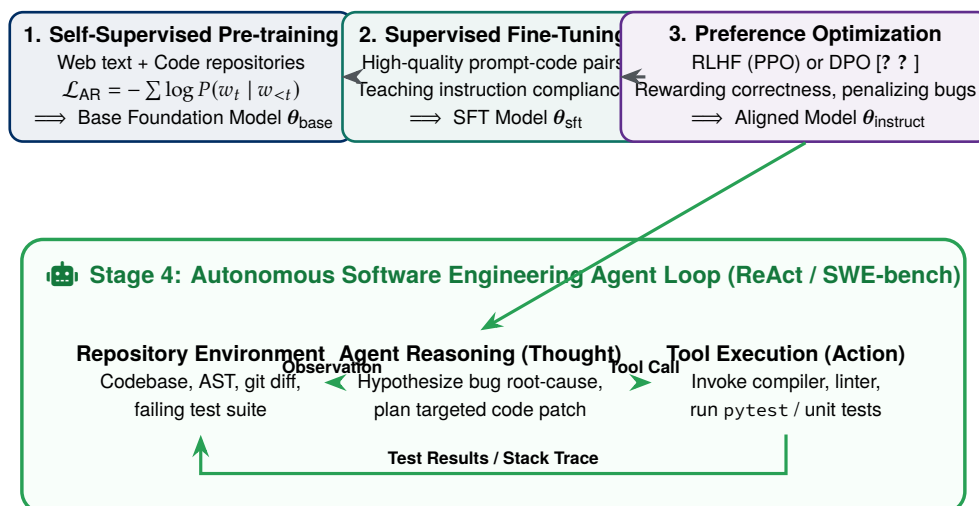


Figure 1.12: End-to-end foundation model lifecycle and autonomous software engineering agent loop.

- 1. Supervised Fine-Tuning (SFT):** The base model θ_{base} is fine-tuned on curated datasets of instruction-response pairs (e.g., bug descriptions paired with git diffs, docstrings paired with function implementations). This conditions the model into an interactive dialogue assistant persona.

2. Preference Alignment (RLHF and DPO):

- *RLHF* (Ouyang et al. 2022 [?]): Employs Proximal Policy Optimization (PPO) to maximize a learned reward function $r_\phi(\mathbf{x}, \mathbf{y})$ subject to a Kullback-Leibler penalty preventing policy collapse:

$$\max_{\theta} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}} \left[r_\phi(\mathbf{x}, \mathbf{y}) - \beta D_{\text{KL}}(\pi_\theta(\cdot | \mathbf{x}) \parallel \pi_{\text{ref}}(\cdot | \mathbf{x})) \right]. \quad (1.52)$$

- *Direct Preference Optimization* (DPO, Rafailov et al. 2023 [?]): Bypasses explicit reward model training by directly optimizing the language model policy on paired preferences ($y_w > y_l$) using the closed-form loss:

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = - \mathbb{E}_{(\mathbf{x}, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | \mathbf{x})}{\pi_{\text{ref}}(y_w | \mathbf{x})} - \beta \log \frac{\pi_\theta(y_l | \mathbf{x})}{\pi_{\text{ref}}(y_l | \mathbf{x})} \right) \right]. \quad (1.53)$$

3. **Test-Time Reasoning Scaffolding:** Wei et al. (2022) [?] demonstrated that prompting models to generate intermediate natural-language reasoning steps (*Chain-of-Thought*) dramatically improves multi-step logic and mathematical reasoning. Modern reasoning models extend this by scaling test-time compute through tree search, self-correction, and iterative verification.
4. **Autonomous Tool-Using Software Agents:** The culmination of modern LLM research is the **ReAct** (Reasoning + Acting, Yao et al. 2022 [?]) paradigm. An autonomous software engineering agent interacts iteratively with a development environment:

$$\text{Observation } o_t \longrightarrow \text{Thought } \tau_t \longrightarrow \text{Action } a_t \in \mathcal{A} \longrightarrow \text{Environment Feedback } o_{t+1}. \quad (1.54)$$

On benchmarks like **SWE-bench** (Jimenez et al. 2024 [?]), which tests whether models can resolve real-world GitHub issues by writing multi-file patches, agents navigate complex codebases, execute terminal commands, run test suites, inspect stack traces, and iteratively refine code patches until all regression tests pass.

Language models have thus transformed from 1948 discrete statistical approximations of character frequencies into neuro-symbolic software engineering collaborators capable of reasoning across repository-scale programs.